

Lecture 30 : Public-key cryptography

(Part II)

Introduction

In the realm of cybersecurity, establishing secure communication channels is paramount. Building upon the foundational concepts of public-key cryptography and the RSA algorithm discussed previously, this lecture delves into alternative and complementary methods for securing digital interactions. While RSA provides a robust framework for encryption and digital signatures, other challenges, such as securely agreeing on a shared secret key over an insecure channel, require different approaches. This session will explore the Diffie-Hellman key exchange, a pioneering public-key algorithm designed specifically for this purpose, and then transition to the critical concept of message authentication, which ensures the integrity and origin of transmitted data. Understanding these mechanisms is crucial for comprehending the multi-layered security architectures that underpin modern digital systems, enabling everything from secure online transactions to private messaging.

Core Concepts

Term	Definition	Significance
Diffie-Hellman Key Exchange	A public-key cryptographic protocol enabling two parties to establish a shared secret key over an insecure communication channel without prior shared secrets.	It allows secure key agreement, which is essential for subsequent symmetric encryption, without directly encrypting data itself.
Discrete Logarithm Problem	The computational difficulty of finding the exponent x given g , n , and $g^x \bmod n$.	The security of the Diffie-Hellman key exchange relies on the assumed computational intractability of this mathematical problem.
Man-in-the-Middle Attack	An active attack where an intruder intercepts and fabricates messages between two communicating	It highlights a significant vulnerability of the basic Diffie-Hellman protocol,

Term	Definition	Significance
	parties, leading them to establish shared keys with the intruder instead of each other.	necessitating additional authentication mechanisms.
Message Authentication	The process by which a receiver verifies the identity of the sender and ensures that the message has not been altered during transmission.	Crucial for data integrity and source verification, preventing impersonation and tampering in communication.
Message Authentication Code (MAC)	A small block of data generated by a secret key and a message, appended to the message to verify its authenticity and integrity.	Provides a cryptographic checksum that, when recomputed and matched by the receiver, confirms the message's origin and lack of alteration.
Cryptographic Hash Function	A mathematical algorithm that takes an input (or 'message') and returns a fixed-size alphanumeric string, called a hash value or message digest.	Used in message authentication to create unique, fixed-size representations of data, making it computationally infeasible to find two messages that hash to the same value.
Symmetric Key Cryptography	Encryption methods where the same secret key is used for both encryption and decryption.	Offers significantly faster encryption/decryption speeds compared to public-key methods, making it suitable for bulk data encryption.
Public Key Cryptography	Encryption methods using a pair of keys: a public key for encryption and a private key for decryption.	Enables secure key exchange and digital signatures, addressing key distribution challenges

Term	Definition	Significance
		inherent in symmetric systems.

Detailed Analysis

1. Diffie-Hellman Key Exchange Protocol

The Diffie-Hellman key exchange is a conceptually simple yet powerful method for two parties, conventionally named A and B, to agree upon a shared secret key over an insecure communication channel. Unlike RSA, it is not designed for encryption or decryption of messages but solely for the secure establishment of a common secret. The security of this algorithm is predicated on the computational difficulty of the discrete logarithm problem.

The protocol unfolds in the following steps:

- * **Agreement on Public Parameters:** Parties A and B first agree on two large public numbers, n (a prime number) and g (a generator, where $g < n$). These values are openly communicated and are not secret; an eavesdropper can know them.
- * **Private Key Generation:**
 - * Party A chooses a large random secret integer, x .
 - * Party B chooses a large random secret integer, y .
- * **Public Value Calculation and Exchange:**
 - * Party A computes $X = g^x \bmod n$ and sends X to B.
 - * Party B computes $Y = g^y \bmod n$ and sends Y to A.
 - * An intruder observing the channel will know n , g , X , and Y . However, due to the discrete logarithm problem, it is computationally infeasible for the intruder to derive x from g , n , and X , or y from g , n , and Y .
- * **Shared Secret Key Computation:**
 - * Party A receives Y from B and computes $K = Y^x \bmod n$.
 - * Party B receives X from A and computes $K = X^y \bmod n$.
 - * Both parties arrive at the same shared secret key K , because $Y^x \bmod n = (g^y)^x \bmod n = g^{(xy)} \bmod n$, and $X^y \bmod n = (g^x)^y \bmod n = g^{(xy)} \bmod n$.
- * **Advantages:** This method requires no prior communication or pre-shared secrets between A and B, making it ideal for initiating secure communication from scratch.
- * **Constraints:** For robust security, the chosen values of n and g must adhere to specific mathematical constraints. For instance, n should be a large prime number, and $(n-1)/2$ should also be prime. Failure to meet such conditions can make the system vulnerable to certain attacks.

2. Man-in-the-Middle Attack on Diffie-Hellman

Despite its elegance, the basic Diffie-Hellman key exchange is susceptible to an active "Man-in-the-Middle" (MitM) attack. This attack occurs when an active intruder (let's call them I) positions themselves between A and B, intercepting and fabricating messages.

The attack scenario unfolds as follows:

- * **Interception:** A initiates communication, sending its public parameters (n , g) and its computed $X = g^x \bmod n$.

- * **Intruder's Deception:**

- * The intruder intercepts A's message.

- * The intruder then acts as A to B. It fabricates its own random secret x_I and sends $X_I = g^{x_I} \bmod n$ to B, pretending it came from A.

- * B, believing it's communicating with A, computes its own $Y = g^y \bmod n$ and sends it back.

- * The intruder intercepts B's Y .

- * The intruder then acts as B to A. It fabricates its own random secret y_I and sends $Y_I = g^{y_I} \bmod n$ to A, pretending it came from B.

- * **Key Establishment with Intruder:**

- * A computes a shared key $K_A = Y_I^x \bmod n = (g^{y_I})^x \bmod n = g^{(xy_I)} \bmod n$. A believes this key is shared with B.

- * B computes a shared key $K_B = X_I^y \bmod n = (g^{x_I})^y \bmod n = g^{(x_I y)} \bmod n$. B believes this key is shared with A.

- * Crucially, the intruder can compute K_A (since it knows x and y_I) and K_B (since it knows x_I and y).

- * **Consequences:** A and B have established shared keys, but A's key is shared with the intruder, and B's key is also shared with the intruder. The intruder can now intercept any encrypted communication between A and B, decrypt it using the appropriate key, read or modify it, and then re-encrypt it with the other key before forwarding it. This compromises both confidentiality and integrity. This vulnerability underscores the need for additional authentication mechanisms to verify the identities of the communicating parties.

3. Comparison of Symmetric and Public-Key Cryptography

A practical comparison between symmetric-key algorithms (like DES or AES) and public-key algorithms (like RSA) reveals significant differences in performance and application:

- * **Speed:** Public-key algorithms are considerably slower. For instance, RSA typically operates at speeds of thousands of bits per second (kilobits per second) for encryption/decryption. In contrast, symmetric-key algorithms like DES can achieve

millions of bits per second, with AES being even faster. This means symmetric-key encryption can be thousands of times faster than RSA.

* **Key Size:**

* RSA offers flexible key sizes, allowing users to choose larger keys (e.g., 2048-bit, 4096-bit) for enhanced security.

* DES has a relatively small fixed key size (56-bit effective).

* AES offers standard key sizes of 128, 192, or 256 bits.

* **Hybrid Approach:** Due to the speed disparity, a common practice in secure communication is to use a two-step, hybrid approach:

1. Public-key cryptography (e.g., RSA or Diffie-Hellman) is used to securely exchange a symmetric secret key.
2. Once the symmetric key is established, symmetric-key cryptography (e.g., DES or AES) is used for the actual bulk encryption and decryption of data, leveraging its superior speed.

4. Message Authentication

Message authentication is a critical security service that ensures the receiver can verify the identity of the sender and confirm that the message has not been tampered with during transit. It addresses the questions of "who sent this?" and "has it been changed?".

Several approaches can be used for message authentication:

* **Conventional Encryption:** Using symmetric encryption can provide some level of authentication if the key is shared only between the sender and receiver, as only the legitimate sender could have encrypted the message.

* **Public-Key Cryptography:** Digital signatures, based on public-key cryptography, offer strong authentication and non-repudiation.

* **Message Authentication Code (MAC):** This method involves generating a small, fixed-size block of data based on the message and a secret key.

* **MAC Workflow:**

1. **Sender:** Takes the message `M` and a shared secret key `k`.
2. **MAC Algorithm:** Runs a MAC algorithm (which can optionally take `k` as a parameter) on `M` and `k` to generate a `MAC` value.
3. **Transmission:** Appends this `MAC` to the message `M` and transmits `(M || MAC)` over the channel.
4. **Receiver:** Receives `(M' || MAC')`.
5. **Verification:** The receiver independently runs the same MAC algorithm on `M'` using the same shared secret key `k` to generate a new `MAC_calculated`.
6. **Comparison:** If `MAC_calculated` matches `MAC'`, the receiver concludes that the message `M'` is authentic and untampered, as only someone with the secret key `k` could have generated the correct `MAC`.

* **Cryptographic Hash Functions:** These functions are often used as a component within MAC algorithms or for digital signatures. They take an input of arbitrary length and produce a fixed-size output (hash value or message digest).

* **Properties:** Cryptographic hash functions are designed to be one-way (hard to reverse), collision-resistant (hard to find two different inputs that produce the same hash), and sensitive to input changes (even a small change in input results in a drastically different hash).

* **Families:** Well-known families include:

* **MD (Message Digest) family:** MD2, MD4, MD5. While historically significant, MD5 is now considered cryptographically broken for many applications due to collision vulnerabilities.

* **SHA (Secure Hash Algorithm) family:** SHA-1, SHA-2 (SHA-256, SHA-512), SHA-3. These offer larger hash sizes (e.g., 160-bit for SHA-1, 256-bit, 384-bit, 512-bit for SHA-2/3) and are generally considered more secure than the MD family.

* **RIPEMD family:** Another family of cryptographic hash functions.

* **Application:** Hash functions are fundamental primitives for building security applications, particularly for ensuring data integrity and as part of digital signature schemes.

Key Takeaways

- [] Diffie-Hellman key exchange is used for securely agreeing on a shared secret key, not for encrypting messages directly.
- [] The security of Diffie-Hellman relies on the computational difficulty of the discrete logarithm problem.
- [] Diffie-Hellman requires no prior shared secrets, only agreement on public parameters n and g .
- [] The basic Diffie-Hellman protocol is vulnerable to an active Man-in-the-Middle attack, where an intruder establishes separate shared keys with each party.
- [] Public-key cryptography (like RSA) is significantly slower than symmetric-key cryptography (like DES/AES).
- [] A hybrid approach, using public-key for key exchange and symmetric-key for bulk data encryption, is common due to performance differences.
- [] Message authentication ensures the receiver can verify the sender's identity and the message's integrity.
- [] Message Authentication Codes (MACs) use a shared secret key to generate a tag that verifies message authenticity and integrity.

- [] Cryptographic hash functions (e.g., SHA family) are essential components for message authentication, providing fixed-size digests that are sensitive to input changes.
- [] Authentication is a fundamental security primitive crucial for building secure applications and organizational network security.

Conclusion

This lecture has illuminated two critical components of modern cryptographic systems: the Diffie-Hellman key exchange and the principles of message authentication. Diffie-Hellman provides an elegant solution for the fundamental problem of establishing a shared secret key over an untrusted channel, a prerequisite for efficient symmetric encryption. However, its susceptibility to Man-in-the-Middle attacks underscores the necessity of integrating additional authentication mechanisms. Message authentication, whether achieved through MACs or cryptographic hash functions, is vital for verifying the origin and integrity of data, thereby combating impersonation and tampering. The understanding of these primitives, their strengths, weaknesses, and their interplay, forms the bedrock for designing robust security applications. As we move forward, the combination of these cryptographic tools will be explored further, demonstrating how they can be integrated to construct comprehensive security services that meet the complex demands of today's interconnected digital world.